Design Doc: redwood-billing-support

`Task.md` is the exhaustive spec (every rule, case, check and decision). This document is the reviewer's version: what the task is, why it is built this way, where it is hard, and why it is fair.

1. What this task is

We have a company called Redwood Inference ,a fictional LLM-inference company which has a lot of messy , raw export of company records across multiple channels(Slack, Gmail, Jira, Confluence, Drive, HubSpot and more). We have a builder agent Claude Opus 5.5 in Claude Code who is tasked with 2 things: 1st build an initial version V0 of a support agent tasked at resolving billing issues for developers using redwood's platform and then as time moves on, the company policies, docs, customers evolve, the builder also has to evolve the billing support agent. Most records have nothing to do with billing, and the billing ones mix current policy with drafts, superseded pages and chatter. Nothing says which records matter. The builder must find the rules and ship an agent and make its different harness components(prompt, rules, skills, tools, hooks, RAG , other components). Then, in the same session, the export grows by 3,547 records: new rules land on top of old ones, three pages are revised in place, some changes apply only from a date onward, and the console's record format changes. The builder must judge which newer record supersedes which older one and evolve its agent without breaking what still applies. The deliverable is the agent itself: both versions hold 120 graded conversations with a scripted developer, and grading reads the billing console's end state and what the agent said, by code, never an LLM judge.

2. Why this design (methodology lineage)

Built this task after reviewing hyper-tau bench, -- hyper tau's benchmark is not at all copied in it, although some of the verifier checks are inspired but the task , environment and checks are made by me.

On reviewing Hyper Tau bench, there were 2-3 fundamental limitation, as more and more humans trust agents , we need to have agents that not only build an agent but also have the ability to evolve them given the change in distribution of data.

Secondly , Hyper Tau Bench doesn't have an exhaustive environment like it doesn't replicates a real enterprise but a real enterprise has messy docs across multiple channels like slack , Gmail etc.

Thirdly, Hyper tau bench only targets prompt, tools, policies whereas an agent has different components and generally support agents have a lot of retrieval/ranking algos which was never touched upon by them

We present a novel task that not only measures how an agent builds another agent through verifier checks like DB end state, Tool actions, NL assertions but also measures the agent's ability to evolve/maintain a agent given changing information

3. The core capability under test

Not "can the model build a RAG agent" (that's a solved capability by itself). The target capability is: when new information lands on top of old, can the agent maintain the agent it built -- decide which new records should change it, change exactly those rules, and keep the old ones that still apply? New is not always current (drafts, chatter), and current is not always total (a $250 limit that applies only to 2027 invoices). An agent that rebuilds from the latest text, or bolts new facts on without scoping them, will look maintained right up until a customer asks about last year's invoice.

4. The designed traps

All of these are real and present in the shipped content (`Task.md` names each record); none is a hidden requirement. They come in four groups: finding the rules, keeping them right when the world changes, applying them in a live conversation, and testing all of that before it ships.

Finding the rules (step 1)

1. **Noise at enterprise scale.** 3,529 records across nine platforms; most are SRE runbooks, sales threads, partner deals and products that have nothing to do with billing. Embedding runs on 2 CPUs and the instruction forbids embedding everything, so the builder must decide what is billing knowledge before it can index it.
2. **Authority, not recency.** Every scored rule has a plausible competitor: a draft price-book plan in Drive, agents' "everything is urgent" chatter in Slack, a provisional Finance x Support meeting, a superseded 2024 thread letting Support grant goodwill under $200, an old RACI draft that routes meter questions to Telemetry, and 14 look-alike pages in the same spaces. Only status, owner and date tell them apart.
3. **Scattered rules.** No record states a whole rule; each piece names the next by key or title. The credits page names Finance's approvals matrix, which holds the limit; the escalation priority is a #support announcement the format page names; which table balances come from is a decision recorded in a Fireflies meeting. A builder that indexes one page per topic gets half a rule.
4. **Inference, not lookup.** The payment-retry date after a processor incident is stated nowhere. Four past status cards fix it uniquely (retries complete 3 business days after the incident window ends), and the agent must work it out for a new incident. *Opus's builds missed it in runs 5, 6 and 9; runs 7 and 8 derived it.*
5. **Restraint is a rule too.** The routing page says to *answer*, not hand off, when the published pages explain what the customer sees (tokenizer counts, billed retries, console lag), "even if the customer calls it a bug". Escalating feels safe and is wrong. *Every Opus build wrote this rule into its own skill. In 23 of 25 conversations (a0-01 and its variants, a0-09-T) the agent still opened a case, usually after retrieving the page in the same conversation; no build practised a token gap.*
6. **Rules can't live in the prompt.** `system_prompt.md` and `AGENTS.md` are capped (1,500 and 4,000 characters) and may not hold exact billing rules, and no store of full source records is allowed. A rule has to become a skill step, a tool behaviour or a retrievable chunk the agent reliably uses: retrieving it is not enough.

Keeping them right when the world changes (step 2)

R1 is not a clean changelog. It mixes six kinds of change, and each kind has its own way to go wrong.

7. **Changed in place.** R1 revises the routing page, the failed-payments page and Finance's approvals matrix at their R0 paths with a later date. A builder that diffs file names, or keeps its R0 index, never sees the change.
8. **Superseded without saying so.** The R0 pages on failed card payments and card updates, credit balances and free-tier limits are replaced by newer ones, but the new pages carry effective dates, not "this replaces X". The old page stays in the export and stays retrievable; only comparing dates and status tells the agent which one governs. The developer often cites the old one ("your 2025 help page says Support refunds duplicates in full"), so the agent must say which page is current rather than follow the customer.
9. **Date-scoped changes.** The duplicate-credit limit drops to $250 only for invoices on the 2027 price book (only Finance's close calendar says that starts 1 March 2027); the usage-delay

threshold becomes 60 minutes only after a cut-over that only a Linear ticket dates. The old rule still governs older invoices, and Q10 replays the R0 conversations to check it. *Opus's A1 dropped the $500 that still applies to 2026 invoices (a0-08 in run 5; a0-07-C and a0-08-T in run 6; a0-07-A and a0-07-C in run 9).*

10. **Contradictions planted next to every change.** Each new rule arrives with a newer-looking competitor: a Slack thread saying the delay threshold is 10 minutes, agents' "everything is urgent" chatter against the announced escalation priority (`normal`), a draft 2027 price-book plan in Drive, and an earlier provisional Finance x Support meeting that disagrees with the recorded decision. 11 more look-alike pages land in the same spaces. New is not the same as current.
11. **Near-miss facts.** The Free-tier quota error is named on the pricing page, but only the 2027 API error reference gives its code (`402 quota_exceeded`); `429`, the per-minute rate limit, is right there beside it. The billing month is now counted in the account's `billing_timezone`, but the format page only defines the field; the rule is in Finance's close calendar. An agent that stops at the first plausible answer gives the wrong one.
12. **New data, new source of truth.** R1 adds a `credit_ledger` table, and a Finance x Support decision says balances now come from it, not from credit grants. The old lookup still returns a number, just the wrong one.
13. **A breaking console change with no policy change ("stale" cases).** R1 serves money as `{value, currency}`, renames the account field on writable tables, adds `billing_timezone` and requires an escalation priority. Cases whose rules never changed (an invoice mismatch, a goodwill request) now fail on A0's tools, so a builder that only reads the new policy text and never re-tests its tools breaks working behaviour.
14. **A procedure withdrawn.** The card-update procedure is retracted in place; card digits must never be repeated (`never_say`), and the developer must be sent to their billing settings instead. Removing behaviour is maintenance too.

Measured together: Q01 grades the unmaintained A0 on the R1 cases (it passes 4 to 8 of 30), so `drift` shows how much of the change A1 absorbed and `regression` shows what it broke. *Opus absorbs most of it (drift 0.75 to 0.96 over runs 5-9) and misses the parts that need judgment: the date scope, and the retry date that no single record states.*

Applying them in a conversation (graded)

15. **Twins.** Five cases per release have a near-identical twin whose correct result is the opposite: only 20 minutes have passed, a self-serve account has no SLA, the duplicate is exactly at the $250 limit, the developer counted only user messages. Matching the ask instead of checking the facts fails one half.
16. **Developers who make it harder.** Each non-twin case has three variants: the deciding facts are missing, the ask is ambiguous, or the developer asserts a false policy. Facts come only through the `ask_developer` form, never in chat, so the agent must ask the right question. *In every Opus run, some a1-07 conversation ends with the agent saying it cannot tell which tokens were the script's, without asking when the script ran (9 of 20).*
17. **Consent and forbidden values.** Credits are graded on order: no credit before the developer's yes. The stale answer is itself checked (`never_say`): the calendar-day retry date, the UTC-month token total.
18. **The conversation's own clock.** Each conversation runs on its own date and time (libfaketime), so "has it been 30 minutes?" and "which month is this?" depend on reading the clock, not the host's.

Verification has to be designed, not borrowed

19. **Five practice conversations per release.** The builder can test its agent on the real deployed model and a practice console, but only five times, and the example scripts cover only the practice accounts. Testing the hard rules means writing scripts for them. *Run 5 practised mostly on the provided examples; none touched a tokenizer gap, a retry date or a scoped limit.*
20. **One session, two deadlines.** Two hours to build, then one hour, in the same Claude Code session, to understand a 3,561-record change to its own system. A0 is frozen and graded as delivered.

For more failure ideas, see `research.md`: failure modes for every part of a support agent's harness (system prompt, AGENTS.md, guardrails and hooks, the agent loop, tools, skills, subagents, retrieval, memory, handoff, budgets). Time allowed only part of it to be built into this task; its last section maps what is implemented to the traps above.

5. Verifier design

5.1 Where it runs and what it writes

`steps/maintain_r1/tests/test.sh` runs after the builder's last step, as root, in the task's own container:

1. `tests/verify.py maintain_r1` checks the delivered workspace and freezes it read-only (A1). `build_r0`'s verifier did the same for A0 at the end of step 1.
2. `/tests` is made root-only, so nothing hidden is readable once graded agents start.
3. `python -m grade.grade` runs the four Q runs below (`GRADE_WORKERS` conversations in parallel, default 4).
4. `python -m grade.final` writes `reward.json` (the trial's reward, which Harbor records) and `scores.json` (the diagnostics), next to every graded conversation under `/logs/verifier/graded/<q>/cases/`.

Q run	Agent	Cases	Console	Feeds
Q00	A0 (the R0 build)	30 R0	R0	`reward`, `q00`, `regression`
Q01	A0	30 R1	R1	`drift` (the unmaintained baseline)
Q10	A1 (the maintained agent)	30 R0	R1 format, R0 records	`reward`, `regression`
Q11	A1	30 R1	R1	`reward`, `q11`, `drift`

5.2 The cases

Each release has 30 cases (`tests/data/billing_scenarios.json`, 60 in all):

- **10 rule cases**, one per scored rule. Every rule cites the builder-visible records that state it in that release (`tests/data/provenance.json`); a test fails if a rule has no visible carrier record.
- **5 twins.** A near-identical ask from another customer whose correct result is the opposite (only 20 minutes have passed; a self-serve account has no SLA; the duplicate is exactly at the limit). An agent that matches the wording of the ask, not the facts, fails one half.
- **15 behaviour variants** of the 5 non-twin cases, one per developer profile:
- `-M` **missing info**: the opening drops the deciding facts; they come only from the developer's form;

- `-A` **ambiguous**: the opening fits several readings; the form or the console settles it;
- `-C` **conflict**: the developer asserts something a published rule or the console refutes (a wrong total, a nonexistent refund term, "your docs say timeouts aren't billed").

A variant keeps its base case's gold exactly (test-enforced), so its base is its control.

Each case also records an **information type**: 1, every deciding fact is in the opening; 2, a deciding fact comes only from the developer when asked; 3, opening facts plus console lookups; 4, facts given only when asked plus console lookups.

5.3 One conversation

- **The developer is a script in code:** an opening, an agenda of follow-ups, facts given only through the `ask_developer` form, one fixed answer to `confirm_with_developer`, and the endings (a stop after the agenda, or a transfer right after a handoff). A model (`USER_MODEL`) only rewords each scripted line as a reply to what the agent just wrote. It never sees the facts, so a question asked in chat goes unanswered, as the agent contract says; a rewording that drops a number or identifier is replaced by the scripted line.
- **The agent runs sandboxed:** as `nobody`, from the frozen workspace, with no network, a 4 GiB address-space limit, a 128 MiB per-file write limit, at most 40 turns (tool rounds included) and 180 seconds per reply, on the conversation's own date and time (libfaketime). Model calls go through the gateway with the run's `SUPPORT_MODEL`.
- **The console** is a fresh, in-process billing store per conversation, holding the case's own release's records in the console release's format, and logging every call.
- **How it ends:** `customer_stop`, `customer_transfer`, `agent_stop`, `max_steps` or `agent_error`. A conversation that did not finish (`max_steps`, `agent_error`) fails every check: its end state is not the agent's answer.

5.4 Checks per conversation

All code, no LLM judge; a conversation passes only if every check it has passes.

Check	Passes when	Example
`db`	the console's writable tables (credit notes, disputes, cases), projected without generated IDs or drafts, equal the gold insert/update trace replayed on a fresh console, or an accepted alternative	a0-01: no case at all; a1-02-T: one $250 credit note on the copy line
`action`	every required console operation succeeded (a query of the right table for the right account, or a write)	look the account up before quoting it
`communicate`	every required value appears in some agent message, in any listed form; case, commas, `$`, markdown and typographic quotes and dashes are normalised, numbers can't run into neighbouring digits, and concepts accept several wordings	"$1,301.46"; "Redwood's tokenizer" or "canonical tokenization"
`never_say`	no forbidden value appears	card digits; the stale "April 10"
`consent`	every credit note or dispute was written after a "yes" from `confirm_with_developer`	

Invalid, not failed. A conversation that says nothing about the agent is excluded from every rate and listed for regrading (`scripts/grade_snapshots.sh`, `--only`): a provider failure that survives LiteLLM's

retries, or a developer leak (the reworded developer said a value the agent must say before the agent did; repeating the agent is not a leak).

5.5 The reward

`reward.json` holds numbers only, each from 0 to 1, higher is better:

Key	Definition
`reward`	the gate: **1.0** if Q00 ≥ 0.9, Q11 ≥ 0.9 and Q10 ≥ Q00 − 0.1; **0.5** if Q00 ≥ 0.9 and Q10 ≥ Q00 − 0.1 (a good build, kept, but R1 not handled); else **0**
`q00`, `q11`	pass rates of the build on R0 and of the maintained agent on R1
`drift`	of the R1 cases the unmaintained A0 fails (Q01), the share A1 passes (Q11): how much of the change maintenance absorbed
`regression`	of the R0 cases A0 passes (Q00), the share A1 still passes (Q10): 1.0 means maintenance broke nothing
`behaviour_robustness`	**user robustness**: over Q00, Q10 and Q11, of the behaviour variants whose base case passed in the same Q run, the share that passed too. A failing base already fails for the rule, so only variants with a passing base measure the developer's behaviour alone. Per profile in `scores.json`
`artifact_quality`	share of the deliverable checks passed: prompt size limits, a non-stub skill, subagent and `@hook`, `load_skill` and `delegate` reachable

A score with nothing to measure (A0 already passes every R1 case, passes no R0 case, or no base passed) is 0, no evidence.

5.6 Diagnostics (`scores.json`, never scored)

Key	What it shows
`q`	the four pass rates, Q01 and Q10 included
`invalid`	per Q, each invalid case and why
`info`	per Q, pass rate by information type, and twin pairs that count only when **both** halves pass (`one_half` lists pairs where only one did)
`behavior`	per Q: pass rate of the bases and of each profile; `robustness` = variant pass rate ÷ base pass rate (above 1 when variants pass where their base failed); and `attribution`, each variant classified against its base: **behaviour failure** (base passed, variant failed), **handled** (both passed), **both failed**, **variant only**
`behaviour_robustness`	the reward's user-robustness score, overall and per profile (`missing_info`, `ambiguous`, `conflict`)
`drift`	the R1 cases A1 absorbed (failed by A0, passed by A1), those not measurable, and `absorbed_by_rule`, absorbed over measurable per rule
`regression`	the R0 cases A1 lost (passed by A0, failed by A1), those A1 improved, and those not measurable
`health`	per Q, **run health**: per-conversation means of wall-clock seconds, model calls, prompt and completion tokens, embedding calls and developer turns, over all cases and over passing cases only (an agent that fails early looks cheap over all cases), and `ends`, a count of how conversations ended, with `agent_error` split into crash or timeout, agent exception, and bad message, and invalid cases counted apart

Key	What it shows
`sizes`	lines per harness layer of A0 and A1 (`system_prompt`, `AGENTS`, skills, subagents, hooks, tools) and the bytes of everything else, mostly the retrieval index
`errors`	any diagnostic that could not be computed, and why

For example, run 8's maintained agent on R1: Q11 health of 32 s, 7.8 model calls, 85K prompt tokens and 2.7 developer turns per conversation, ending in 19 stops and 11 transfers; behaviour robustness 0.97 overall (missing info 0.92: one variant, a1-07-M, failed where its base passed); sizes growing from 172 to 211 hook lines and 279 to 350 tool lines between A0 and A1.

5.7 Anti-cheating

The builder never sees `/tests`: Harbor copies it in only at verification, and it is made root-only before any graded agent runs. Hidden cases, the console seeds and internals, and the R1 overlay are root-only in the image, and no test file is copied into the image. During the build only the practice sidecar holds provider keys; the graded agent runs with no network and no credentials. No hidden case's account, invoice or request identifier appears in the builder-visible corpus.

5.8 How the verifier is tested

`dev/tests` (183 tests) cover the verifier itself: every gold trace replays and passes; a do-nothing agent fails all 60 cases; each twin's gold fails its pair; each R1 case fails the write its R0 rule would make; variants keep their base's gold and each profile appears five times per release; correct explanations in other words pass while vague or echoed text fails; a value cannot hide inside an identifier (`req_pb402` would have passed "402"); Q10 conversations see only their own release's records; provider failures become invalid cases; the reward keys and the gate; and the practice service matching the graded loop.

6. Oracle

The oracle (`steps/*/solution/solve.sh`) is a reference build, not a shortcut: it installs a workspace and then runs through exactly the same conversation loop, sandbox, console, support model and checks as the builder's agent.

- `build_r0`**:** installs the reference workspace and builds its index (`build_index.py`) over the billing-relevant records with the same local embedder. Its parts: hybrid retrieval (`retrieval.py`: LanceDB full-text and dense search merged by reciprocal-rank fusion, capped per source); a `billing-investigation` skill with a step per kind of request (what to look up, which current rule to search for, when a handoff is allowed); a `policy-researcher` subagent; hooks that block a write before the account is looked up and an escalation that does not quote a retrieved routing rule naming the team; and `billing_rules.py` for the duplicate-credit limit.
- `maintain_r1`**:** overlays only what R1 changed and embeds only the records R1 added or revised (`build_index.py --new`). The changes are the maintenance this task asks for: `billing_rules.py` scopes the duplicate-credit limit by invoice date (`duplicate_credit_limit_cents(period_start)`: $250 from March 2027, $500 before); the skill gains steps for card-payment failures, the credit ledger, the account's billing time zone and the routing page's new missing-usage threshold; the hooks gain card-number redaction and consent before a credit or dispute. The session record is continued, as the builder's must be.

It scores reward 1.0 in 6 of 6 runs (`evidence/oracle-submitted` on exactly the submitted code), missing at most a case or two per run to the support model's sampling, a different case each time.

7. Fairness audit against the assignment rubric

- **Solvable.** The oracle (`steps/*/solution/solve.sh`) builds a workspace through the same runtime, sandbox and console the builder's agent uses, and scores reward 1.0 in 6 of 6 runs (Q00 30 of 30 every time, Q11 28 to 30 of 30). Every rule a case tests cites builder-visible records for its release (test-enforced). Every value the agent must say appears in a tool result, except the retry date, which the status cards determine uniquely.
- **Unambiguous.** The instructions state the deliverable, the interfaces, the limits and how success is measured. The agent contract documents every tool and field the developer form accepts. Gold actions and accepted alternatives cover every outcome the records allow (e.g. either duplicated line).
- **Substantive, long-horizon.** Corpus forensics, then architecture and indexing, then tools, skills, subagent and hooks, then practice and debugging; then diffing a changed corpus, re-indexing, scoped edits and regression checks. Two steps, about 3 hours, one session.
- **Reproducible.** Pinned base image digest, a full `requirements.lock`, an embedding model baked into the image, and fixed data. Model access is declared: the builder's harness, and one provider key for the deployed support model (Section 8).
- **Non-brittle.** Every check is code over the console state or normalised text (case, commas, `$`, markdown, quotes and dashes ignored; any listed form counts). No exact-string grading of prose. Invalid cases (provider failure, a developer wording that leaks an answer) are excluded, not failed.
- **No leakage.** `/tests` arrives only at verification and is made root-only before graded agents run; hidden data, the R1 overlay and console internals are root-only in the image; during the build only the practice-model container holds the provider key. The builder traces of all five Opus runs were audited: no hidden path, value, network fetch or R1 record reached the builder before R1. An earlier grading-time leak (task folder world-readable) was found and closed; no workspace had used it.
- **The simulated developer is fair.** Audited over Opus runs 5-9 and the oracle (about 1,270 worded lines): no team or rule names, no answers to questions asked in chat, no required value said before the agent said it, and endings always match the console; the few lines with an extra value repeat what the agent had just said.
- **Every failure is the agent's.** Each failing conversation of Opus runs 5-9 was read against its records: every deciding fact is builder-visible and published (or, for the retry date, uniquely determined by published status cards), and the oracle passes each of those cases in at least 4 of the 5 curated oracle runs. Four verifier or case defects the audit found (one ambiguous case, a hidden requirement, one-wording phrase matching, newer invoices visible to older conversations) were fixed with tests, and all five Opus runs still score reward 0 when rescored (`RUN_REPORT.md`, *Fairness audit*).
- **Net-new.** The scenario, rules, console, cases, runtime and verifier were written for this task. The only external content is the MIT background corpus and public libraries (`README.md`, *Provenance*).

8. Reproducibility

8.1 Requirements

Requirement	Specification
Harness	Harbor 0.22.0 (`harbor run`), Claude Code agent `claude-code`

Requirement	Specification
Container runtime	Docker with Compose v2; images are Linux and multi-arch (verified on Apple Silicon with Colima)
Resources per trial	2 CPUs, 8 GiB memory, 20 GiB storage for the main container (`task.toml`), plus a small `practice-model` sidecar
Disk	about 2 GB per built image and up to 1 GB of job output per trial
Builder credentials	`CLAUDE_CODE_OAUTH_TOKEN` (subscription) or `ANTHROPIC_API_KEY`; for another Anthropic-compatible endpoint, `ANTHROPIC_BASE_URL` plus its key (run 9 uses OpenRouter this way)
Support-model credentials	`SUPPORT_MODEL` and `USER_MODEL` (LiteLLM ids, e.g. `openai/gpt-6-luna`) and the provider key LiteLLM reads for them: `OPENAI_API_KEY` and `OPENAI_BASE_URL`, `ANTHROPIC_API_KEY` or `OPENROUTER_API_KEY`
Provider rate limit	about 1M tokens per minute at the default `GRADE_WORKERS=4`, about 500K at 2 (120 conversations of about 7 calls each)
Network	allowlisted hosts only (`task.toml`): Claude Code's install and API hosts for the builder; the support model's provider hosts for the practice sidecar and the grading step
Wall-clock time	about 3 h of building (2 h and 1 h steps) plus 1-2 h of grading inside `maintain_r1`'s verifier (`timeout_sec = 10800`)

All secrets are read from the host environment (`${VAR}` in `task.toml` and `docker-compose.yaml`); none is stored in the task. During the build, only the `practice-model` container receives provider keys.

8.2 What is pinned

- **Base image** by digest (`python:3.12-slim@sha256:…`).
- **Python dependencies** by exact version in `environment/requirements.lock`, including LiteLLM 1.104.2.
- **Data**: the corpus (R0, R1), the billing console seeds and the 60 cases are files in the task.
- **Embeddings**: `sentence-transformers/all-MiniLM-L6-v2`, downloaded once at build time and run locally; no embedding API is used.
- **The support model** is fixed per run by `SUPPORT_MODEL` and written into the builder-visible manifest.

8.3 What is not pinned

- **Debian packages** (`nodejs`, `npm`, `libfaketime`, `procps` and a few tools) are installed from the distribution at build time.
- **The embedding weights** are fetched from Hugging Face during the build.
- **Claude Code** is installed by Harbor at run time from Anthropic's release host.

9. Economic viability

- **Builder:** on average **$16.22 per trial** ($14.82-17.21): about $5.50-6 for the R0 build and $9.30-10.50 for maintenance, by Claude Code's own cost meter, which reports the API-equivalent cost.
- **Support model (gpt-6-luna):** on average **$0.67 per trial** for the 120 graded conversations of an Opus build ($0.54-0.76), and about $0.35 for the oracle's, from the token counts recorded on every graded model call (about 12-17M input tokens per trial, 70-82% of them cache hits, and 0.3-0.4M output tokens) at published gpt-6-luna prices ($0.10/M input, $0.01/M cached input,

$0.50/M output). The developer's wording and the builder's practice conversations add a few cents. A Claude-class support model would cost far more, which is why gpt-6-luna is the default.

- **Total:** about **$17 per trial**, almost all of it the builder.
- **Why it is worth running.** Support agents grounded in company knowledge are among the most deployed LLM applications, and their running cost is upkeep: pricing, policy and data formats change, and someone has to update the agent without breaking it. A wrong answer here is a real cost (an unneeded engineering ticket, a credit over the approval limit, a wrong retry date given to a customer). The task measures that upkeep directly, with a cost per trial that allows repeated runs.